

Course Code: SE-3003

Course: Web Engineering Lab

Spring 2025, Lab Manual – 03

User define functions

JavaScript Functions

A JavaScript function is a block of code designed to perform a particular task. A JavaScript function is executed when "something" invokes it (calls it).

```
// Function to compute the product of p1 and p2
```

```
function myFunction(p1, p2) {  
    return p1 * p2;  
}
```

JavaScript Function Syntax

A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses (). Function names can contain letters, digits, underscores, and dollar signs (same rules as variables). The parentheses may include parameter names separated by commas: (parameter1, parameter2, ...)

The code to be executed, by the function, is placed inside curly brackets: {}

Function Return

When JavaScript reaches a return statement, the function will stop executing. If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

Functions often compute a return value. The return value is "returned" back to the "caller":

```
let x = myFunction(4, 3); // Function is called, return value will end
```

up in x

```
function myFunction(a, b) {  
    return a * b; // Function returns the product of a and b  
}
```

DOM:

With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

The HTML DOM (Document Object Model, When a web page is loaded, the browser creates a Document Object Model of the page. The HTML DOM model is constructed as a tree of Objects:

With the object model, JavaScript gets all the power it needs to create dynamic HTML: JavaScript can change all the HTML elements in the page

- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes JavaScript can add new HTML elements and attributes JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

```
<div id="a"></div>
```

```
<div class="c"></div>
```

```
<h1> </h1>
```

```
<p class="intro">Hello World!.</p>
```

```
<h4 id="demo"> </h4>
```

```
<script>
```

```
document.getElementById("a").innerHTML = "Its is just DOM by ID";
```

```
document.getElementsByClassName("c")[0].innerHTML = "Its is just DOM by Class";
```

```
document.getElementsByTagName("h1")[0].innerHTML = "Its is just DOM by ID";
```

```
const x = document.querySelectorAll(".intro");
```

```
document.getElementById("demo").innerHTML =
```

```
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;
```

```
</script>
```

```
<h3 type="text" name="naam">This is Tag Name</h3>
```

```
<h3 type="text" name="naam">This is Tag Name</h3>
```

```
<input type="button" onclick="Dom()" value="Click here" />
```

JavaScript Events

HTML events are "things" that happen to HTML elements. When JavaScript is used in HTML pages, JavaScript can "react" on these events.

HTML Events

An HTML event can be something the browser does, or something a user does. Here are some examples of HTML events. An HTML web page has finished loading. An HTML input field was

changed. An HTML button was clicked Often, when events happen, you may want to do something. JavaScript lets you execute code when events are detected. HTML allows event handler attributes, with JavaScript code, to be added to HTML elements. With single quotes:

```
<element event='some JavaScript'>
```

```
<element event="some JavaScript">
```

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The time  
is?</button>
```

Event Validation Example:

```
<html>
```

```
<script>
```

JavaScript Loops

Loops are handy, if you want to run the same code over and over again, each time with a different value. Often this is the case when working with arrays:

- for - loops through a block of code a number of times
- for/in - loops through the properties of an object
- for/of - loops through the values of an iterable object
- while - loops through a block of code while a specified condition is true
- do/while - also loops through a block of code while a specified condition is true

The syntax of these loops are

```
for (expression 1; expression 2; expression 3)
```

```
{
```

```
// code block to be executed
```

```
}
```

```
for (key in object)
```

```
{
```

```
// code block to be executed
```

```
}
```

```
for (variable of iterable) {
```

```
// code block to be executed
```

```
}
```

```
while (condition) {
```

```
// code block to be executed
}

// for loop
<script>
for(let i = 0;i<10;i++){
    document.querySelector("#d").innerHTML+=`<p style="background-
color:blue;color:white">${i}</p>`;
}
</script>
```

Code snap

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Document</title>

</head>

<body>

    <p> <b> Select your gender: </b> </p>

    <input type="radio" id="g1" name="gender" value="male">

    <label for="male"> Male </label> <br>

    <input type="radio" id="g2" name="gender" value="female">

    <label for="female"> Female </label> <br>

    <input type="radio" id="g3" name="gender" value="other">

    <label for="other"> Other </label>

    <br>

    <input type="submit" value="Submit" onclick=" checkRadioVal()">

<p id="demo"></p>

<script>

    function checkRadioVal(){

let radio = document.querySelectorAll('input[name="gender"]');
```

```

for(let x=0;x<radio.length;x++){
    if(radio[x].checked==true){
demo.innerHTML = `you are selected ${radio[x].value} gender`; }
    }

}

</script>
</body>
</html>

```

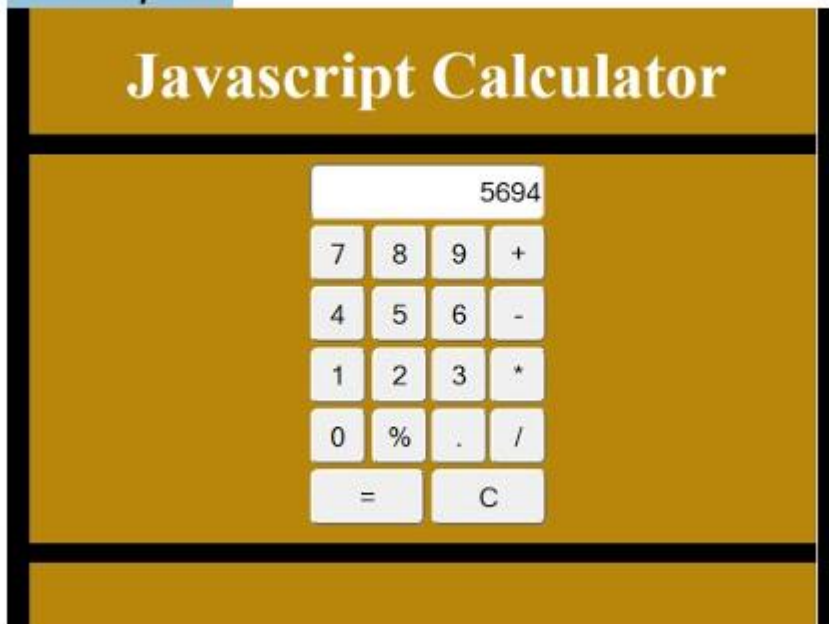
Task 1

Write a JavaScript program that takes a number input from the user and displays its multiplication table in the web browser. The table should be in HTML table format, with even rows having a background color of #BAD8B6 and odd rows having a background color of #638C6D. The text color should be white

Task 2

Create a JavaScript program to display the simple Calculator which will perform basic arithmetic operations.

Example:



Task 3:

Validation is one of the important concept in the web page, create a complete Sing up page having name, contact, email password, radio and checkbox button on it and validate using JS regular expression individually according to the user control.

Tailwind CSS

Introduction To Tailwind CSS

Tailwind CSS is an open-source CSS framework that provides utility classes for styling your website directly in your HTML. Unlike traditional frameworks like Bootstrap, it doesn't offer pre-built components, but a large collection of low-level utility classes like ***text-blue-500, w-full, or flex-row.***

How it works:

You add these classes to your HTML elements, and Tailwind translates them into the corresponding CSS styles when you build your website. This allows for rapid development and a clean, consistent user interface.

Benefits:

- **Speed:** Build UIs quickly without writing much custom CSS.
- **Flexibility:** Highly customizable to achieve any design.
- **Maintainability:** Easier to read and maintain HTML with clear class names.
- **Responsiveness:** Responsive styles built-in for various screen sizes.

Drawbacks:

- **Learning curve:** Takes time to learn the class names and syntax.
- **Verbosity:** Can lead to HTML files with many class names.
- **Limited customization:** Not ideal for creating highly unique UI elements.

How To Setup Tailwind CSS:

There are two ways to use Tailwind CSS in Your html document.

1. Online Through CDN Link
2. Offline Use Through Setting up Tailwind offline for project.

- **Online Through CDN Link**

Include the Tailwind CSS CDN link in your HTML's <head> section:

```
<script src="https://cdn.tailwindcss.com"></script>
```

Demo Code Given Below

```
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>
  <h1 class="text-3xl font-bold underline bg-red-600 text-center mt-10">
    Welcome to Tailwind CSS
  </h1>
</body>
</html>
```

- **Setting up Tailwind offline for project**

1. For Setting up Tailwind CSS offline you must have Node.js Installed on your system
2. Link To download Node JS (Download and Install the latest version from the below link [Node.js \(nodejs.org\)](https://nodejs.org))
3. Create a Folder (E.g. TailwindCSS) and open your folder in vs code.
4. Open the Terminal in Your VS code and Initialize your project using ***npm init -y*** This will create the Package.json file in your folder.
5. Create Public Folder inside Your Main Folder.
6. Inside Public Folder Create your index.html file.

7. Now Install the Tailwind using **npm install tailwindcss @tailwindcss/cli** (run this on terminal) this will create package-lock. Json. **And dependencies "dependencies": {**
"@tailwindcss/cli": "^4.0.3",
"tailwindcss": "^4.0.3"
} in package.json file
8. Create **src** folder and **input.css** file in main directory.
9. Add the **@import "tailwindcss";** import to **input.css** file
10. Run **npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch**
11. Now for initializing Tailwind CSS run **npx tailwindcss init** on terminal this will create tailwind.config.js.
12. Now we need to configure which files we need to use tailwindcss for that we have to write the location of our files and folders in the content inside tailwind.config.js.
13. Suppose I want to use it for html files inside the public folder so I will write
content: ["/Public/*.html"],
14. Now Create a New Folder src in your main folder
15. Create input.css file inside src folder and paste these lines.
@tailwind base;
@tailwind components;
@tailwind utilities;
16. Now link your output.css with html files (i.e index.html)

- **Tip for Tailwind CSS Class**

You can install the extension Tailwind CSS IntelliSense in your vs code to get the help of tailwind classes.

Getting Started with Tailwindcss Classes

1. Core Utility Classes:

Tailwind CSS takes a unique approach to styling your web projects, offering a vast collection of pre-defined classes instead of traditional CSS. Think of these classes as tiny building blocks, each performing a specific styling task like setting margin, color, or font size. Unlike custom CSS, you combine these utility classes directly in your HTML, reducing repetitive code and speeding up development.

Common Tailwind Utility Classes and Examples

Utility Class	Description	Example
Spacing		
m-x	Margin on all sides (horizontal)	<code><div class="m-4">Content with 1rem margin around</div></code>
my-y	Margin on top and bottom (vertical)	<code><p class="my-2">Paragraph with 0.5rem top and bottom margin</p></code>
p-x	Padding on all sides (horizontal)	<code><button class="p-4">Button with 1rem padding</button></code>
py-y	Padding on top and bottom (vertical)	<code><h2 class="py-3">Heading with 1.5rem top and bottom padding</h2></code>
Sizing		
w-x	Width	<code></code> (Full-width image)
h-y	Height	<code><iframe class="h-64" src="video.mp4"></iframe></code> (64px tall iframe)
min-w-x	Minimum width	<code><input class="min-w-full" type="text"></code> (Input field with minimum full width)
max-h-y	Maximum height	<code><div class="max-h-48">Scrollable content with max height of 48px</div></code>
Text & Color		
text-x	Text color	<code><h3 class="text-blue-500">Blue heading</h3></code>
font-weight	Font weight	<code>Important text</code>
text-size	Text size	<code><p class="text-2xl">Large paragraph text</p></code>
bg-x	Background color	<code><div class="bg-gray-200">Content with background color</div></code>
Layout & Display		
flex	Makes element a flexbox container	<code><div class="flex">Flexbox with child elements</div></code>
grid	Makes element a grid container	<code><main class="grid grid-cols-3">Three-column grid layout</main></code>
block	Makes element display as block-level	<code>Block-level button</code>

inline	Makes element display as inline	<code>Inline text</code>
Positioning		
absolute	Positions element relative to its nearest positioned ancestor	<code><div class="absolute top-0 right-0">Absolutely positioned element</div></code>
relative	Establishes positioning context for its children	<code><div class="relative">Element with absolutely positioned children</div></code>
fixed	Positions element relative to the viewport	<code><nav class="fixed top-0 left-0">Fixed navigation bar</nav></code>

2. Customization:

Tailwind's beauty lies in modifying base classes using various suffixes. For example, the `text-gray-700` class sets the text color to a specific gray shade. Modify it with:

Responsiveness: Add suffixes like `sm:`, `md:`, `lg:`, etc., to apply styles only at specific screen sizes (e.g., `text-gray-700 md:text-gray-800` changes text color on medium screens and above).

Hover, Focus, Active States: Use `.hover`, `.focus`, and `.active` to define styles for different interactions (e.g., `button:hover:bg-blue-500` changes button background on hover).

Variants: Apply variations like `opacity-50` for transparency, `rounded-full` for circular elements, and more, directly after the base class (e.g., `text-gray-700 font-bold opacity-75`).

Customizing Values:

Each utility class often accepts numerical values (e.g., `px` for spacing, `rem` for font size). While Tailwind provides pre-defined options, you can override them:

Use inline styles directly after the class (e.g., `p-4 px-8` sets both horizontal and vertical padding, but with different values).

In the `tailwind.config.js` file, adjust the default values for specific utilities by changing the corresponding array in the config section.

Creating Custom Utility Classes:

When specific styles appear frequently, create custom classes for organization and efficiency: Define a `.card` class in your CSS with desired margin, padding, and background:

```
.card {  
  margin: 1rem;  
  padding: 2rem;  
  background-color: #f5f5f5;  
}
```

Use it throughout your HTML as needed:

HTML

```
<div class="card">  
  
</div>
```

Mastering Responsiveness

Tailwind CSS shines in its approach to responsive design, offering built-in tools and modifiers to seamlessly adapt your website across different screen sizes. Here's a deep dive into how it empowers responsive development:

1. Mobile-First Philosophy:

Tailwind adopts a mobile-first approach, meaning styles are applied by default to mobile devices (small screens).

As screen sizes increase, you add responsive modifiers to target larger devices (e.g., `md:`, `lg:`, `xl:`) and adjust styles accordingly. This ensures your website looks its best on mobile first, then gracefully scales up for larger screens.

2. Responsive Modifiers:

Each utility class comes with responsive variants denoted by suffixes like `md:`, `lg:`, `xl:`, etc.

These modifiers let you apply different styles at different screen widths.

For example, you can use `p-4 md:p-8` to set padding of 1rem on all sides for small screens, but increase it to 2rem on medium and larger screens.

3. Breakpoints:

You can customize the screen sizes at which responsive modifiers take effect by adjusting Tailwind's pre-defined breakpoints in the `tailwind.config.js` file. This allows you to tailor the responsiveness to your specific design needs and target devices you prioritize.

To Customize do the following.

Open `tailwind.config.js` in your preferred text editor or IDE.

Search for the `screens` property within the theme object. It's often found under the `extend` section, like this:

JavaScript

```
module.exports = {  
  theme: {  
    extend: {  
      screens: {  
        'sm': '640px',  
        'md': '768px',  
        'lg': '800px', // New breakpoint for two cards  
        'xl': '1280px',  
        '2xl': '1536px',  
      }  
    }  
  }  
};
```

4. Utility-First Styling:

Unlike frameworks with pre-built components, Tailwind uses low-level utility classes, giving you granular control over responsiveness. You can combine and modify these classes to achieve intricate, device-specific layouts and styles.

5. Responsiveness in Action:

```
<!doctype html>  
<html>  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
  <link rel="stylesheet" href="/output.css">  
</head>  
<body class="bg-slate-900">
```

```
<p class="sm:text-sky-500 md:text-green-600 lg:text-red-700 text-white">
```

Lorem ipsum dolor sit amet consectetur adipisicing elit. Voluptatum doloribus ipsum, nesciunt, odit, maiores consequatur sed est temporibus rerum vel nihil perferendis veniam aliquam! Distinctio quas aliquid beatae officia corrupti.

</p>

</body>

</html>

The provided HTML uses Tailwind's responsive modifiers to change the text color based on screen size:

sm:text-sky-500: Sets light blue on small screens.

md:text-green-600: Overrides to dark green on medium screens.

lg:text-red-700: Overrides again to dark red on large screens.

text-white: Acts as the fallback for all other screen sizes, using white text.

This code achieves responsive text color adjustments using Tailwind's built-in screen adaptation without requiring additional media queries or custom CSS.

Some Example Code Snippets

Navbar with Tailwind

```
<!doctype html>
```

```
<html>
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>try new</title>
```

```
  <link rel="stylesheet" href="./output.css">
```

```
</head>
```

```
<body class="">
```

```
  <nav class="bg-blue-500"> <div class="max-w-7xl mx-auto px-2 sm:px-4">
```

```
    <div class="flex items-center justify-between h-16">
```

```
      <div class="flex items-center">
```

```
        <a href="#" class="flex-shrink-0">
```

```
          
```

```
        </a>
```

```
      </div>
```

```

<div class="hidden sm:block">
  <a href="#" class="ml-auto px-3 py-2 text-white font-medium">Home</a> <a href="#" class="ml-4 px-3 py-2 text-white font-medium">About</a>
  <a href="#" class="ml-4 px-3 py-2 text-white font-medium">Services</a>
  <a href="#" class="ml-4 px-3 py-2 text-white font-medium">Contact</a>
</div>

<div class="-mr-2 flex items-center sm:hidden">
  <button type="button" class="relative inline-flex items-center justify-center rounded-md p-2 text-gray-400 hover:bg-gray-700 hover:text-white focus:ring-2 focus:ring-white focus:outline-hidden focus:ring-inset" aria-controls="mobile-menu" aria-expanded="false">
    <span class="sr-only">Open menu</span>
    <svg class="h-6 w-6" xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke="currentColor" aria-hidden="true" data-slot="icon">
      <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 6h16M4 12h16M4 18h16" />
    </svg>
  </button>
</div>
</div>
</div>
</nav>

```

```

<div class="sm:hidden bg-amber-950 hidden" id="mobile-menu">
  <div class="px-2 pt-2 pb-3 space-y-1">
    <a href="#" class="block px-3 py-2 text-base font-medium text-black">Home</a>
    <a href="#" class="block px-3 py-2 text-base font-medium text-black">About</a>
    <a href="#" class="block px-3 py-2 text-base font-medium text-black">Services</a>
    <a href="#" class="block px-3 py-2 text-base font-medium text-black">Contact</a>
  </div>
</div>

```

```

<script>
document.addEventListener('DOMContentLoaded', () => {
  const toggleButton = document.querySelector('[aria-controls="mobile-menu"]');
  const mobileMenu = document.getElementById('mobile-menu');

```

```
toggleButton.addEventListener('click', () => {  
    mobileMenu.classList.toggle('hidden');  
  
    });  
});  
</script>  
</body>  
</html>
```

Grid Example Code With cards.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>card</title>  
    <link rel="stylesheet" href="output.css">  
</head>  
<body>  
  
    <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4">  
        <div class="card bg-white rounded-lg shadow-md p-4 ">  
              
            <div class="p-4 ">  
                <h3 class="text-xl font-bold text-gray-700">Card Title 1</h3>  
                <p class="text-gray-600">Card description goes here.</p>  
  
            </div>  
        </div>  
  
    </div>  
  
    <div class="card bg-white rounded-lg shadow-md p-4">
```

```

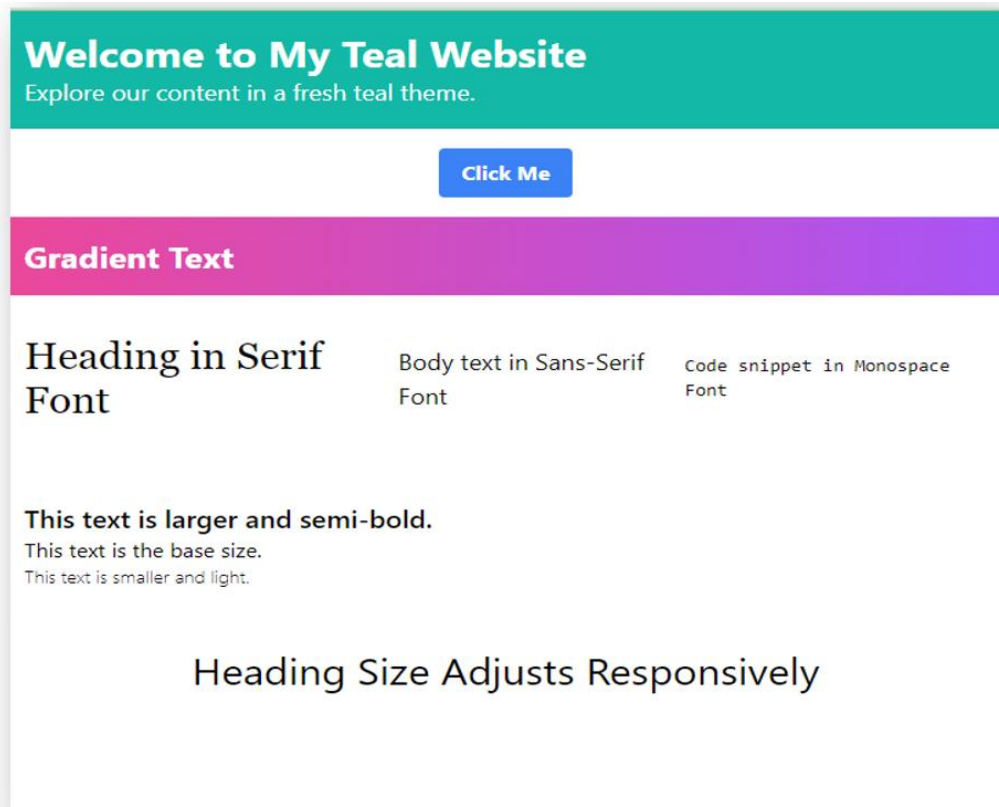

<div class="p-4">
  <h3 class="text-xl font-bold text-gray-700" >Card Title 2</h3>
  <p class="text-gray-600" >Card description goes here.</p>
</div>
</div>

<div class="card bg-white rounded-lg shadow-md p-4">
  
  <div class="p-4">
    <h3 class="text-xl font-bold text-gray-700">Card Title 3</h3>
    <p class="text-gray-600">Card description goes here.</p>
  </div>
</div>

<div class="card bg-white rounded-lg shadow-md p-4">
  
  <div class="p-4">
    <h3 class="text-xl font-bold text-gray-700">Card Title 4</h3>
    <p class="text-gray-600">Card description goes here.</p>
  </div>
</div>
</div>

</body>
</html>
```

Task-1: Using Tailwindcss perform following Tasks



Task2

Perform The following.

- Create a basic Registration form!
- The input border should change color to red when it is focused.
- The form not be visible in screen below medium and below devices instead it will show kindly change the device to see the form.
- Button color should be black and text white on hover color will change to Green and white
- Form Background and text color should change when the screen size change

Task 3

- Colorful Unordered List: Craft an unordered list with custom bullet colors (e.g., red, teal) and contrasting text for easy reading.
- Numbered Ordered List: Build an ordered list with custom starting numbers (e.g., list-decimal-5) and explore alternative numbering styles (e.g., roman, lower-roman).
- Indented Nested Lists: Create nested lists with proper indentation using classes like ml-4 and style nested items differently for clarity.
- Iconic Description List: Combine the dl element with icons for terms, customize their colors and sizes, and add clear definitions using dd.
- Horizontal List with Spacing: Construct a horizontal list using flex, space items with mr-4/ml-4, and style them as buttons or links for interactivity.

Task4

- Do the following Responsive Image Gallery:
- Maintain the original grid layout and image classes for the Image gallery.
- Consider using media queries to adjust h-48 sm:h-64 md:h-80 for different screen sizes, though it won't offer the same dynamic resizing as in the original code.
- 2. Product Image with Zoom on Hover:
- Use CSS transformations (e.g., hover: scale-110) and transitions (e.g., transition duration-200) to achieve a basic zoom effect without JavaScript.

Task 5

Create the following using tailwindcss

